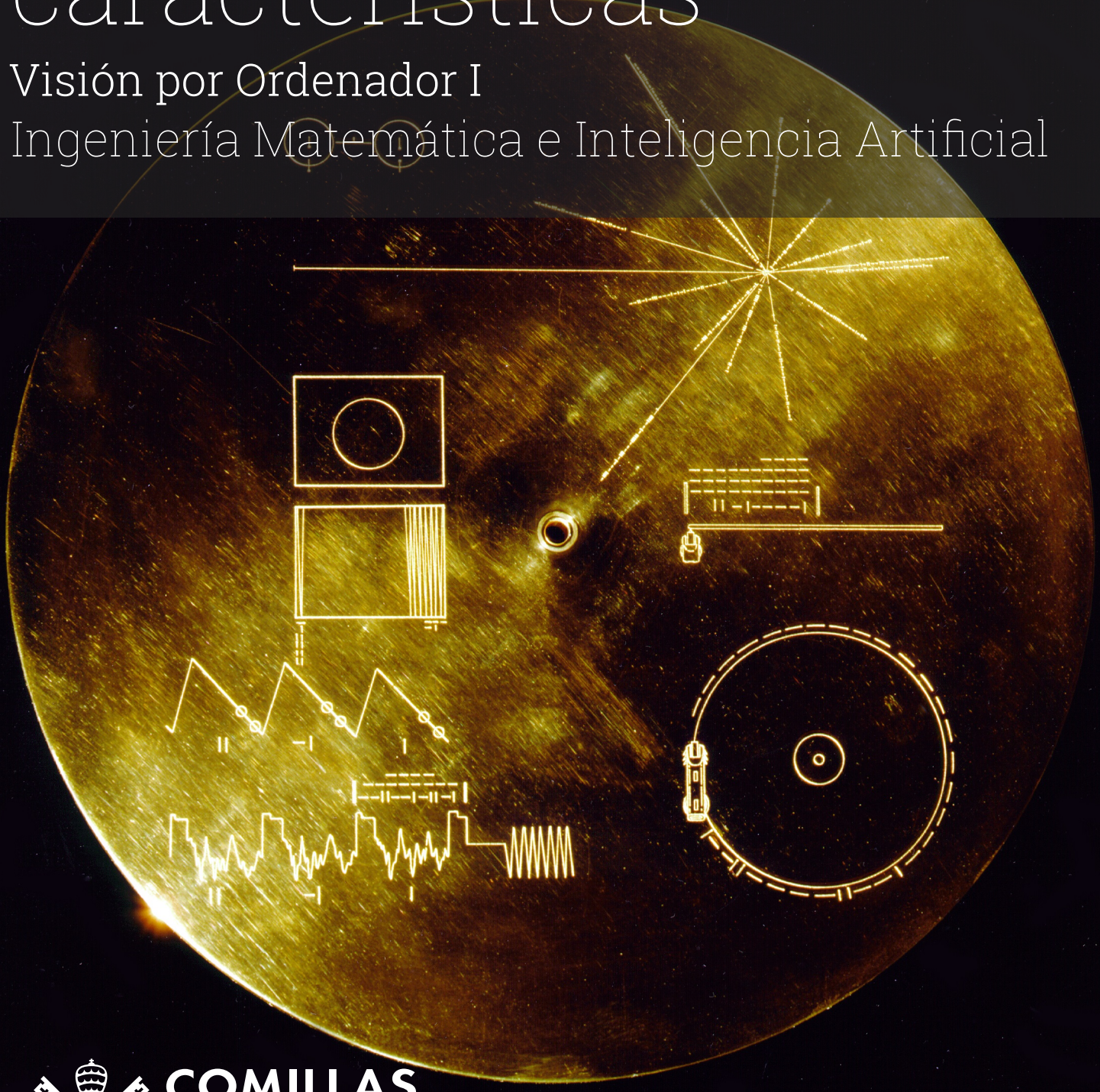
Laboratorio
Extracción de características

Visión por Ordenador I

Ingeniería Matemática e Inteligencia Artificial



Universidad Pontificia Comillas, ICAI

COMILLAS
UNIVERSIDAD PONTIFICIA
ICAI

3º Ingeniería Matemática e Inteligencia Artificial

Curso 2026/27

Laboratorio
Extracción de características

Profesor:	**Email**
Jaime Boal Martín-Larrauri	jboal@comillas.edu
Lionel Güitta	lglopez@comillas.edu
Daniel Pinilla	dpinilla@comillas.edu
Luis Arias	learias@comillas.edu
José María Bengochea	jmbengochea@comillas.edu
Mario Triviño	mtrivino@comillas.edu

Cover: Flying aboard Voyagers 1 and 2 are identical "golden" records, carrying the story of Earth far into deep space. This gold aluminum cover was designed to protect the gold-plated records from micrometeorite bombardment, but also serves a double purpose in providing the finder a key to playing the record. Image Credit: NASA/JPL-Caltech



COMILLAS
UNIVERSIDAD PONTIFICIA
ICAI

Contents

1 Sesión 4: Extracción de características **1**
1.1 Materiales . . . 1
1.2 Apartados de la práctica . . . 1
1.3 Observaciones . . . 1
1.4 Qué va a aprender . . . 2
1.5 Evaluación . . . 2

2 Apartado A: Detección de esquinas **3**
Tarea A.1: Creación de carpeta de resultados . . . 3
Tarea A.2: Carga de imágenes . . . 3
Tarea A.3: Definición del método de Harris para detección de esquinas . . . 3
Tarea A.4: Aplicación de Harris . . . 3
Tarea A.5: Definición del método de Shi-Tomasi para detección de esquinas . . . 4
Tarea A.6: Aplicación de Shi-Tomasi . . . 4
Preguntas . . . 4

3 Apartado B: Detección de líneas rectas **5**
Tarea B.1: Aplicación de Canny . . . 5
Tarea B.2: Visualización de líneas . . . 5
Tarea B.3: Aplicación de la transformada de Hough . . . 5
Preguntas . . . 6

4 Apartado C: Detección de puntos de interés y bolsa de palabras **7**
Tarea PREVIA: Creación de las carpetas de resultados . . . 7
C.1: Detección de puntos de interés . . . 7
Tarea C.1.1: Generar función de filtro gaussiano . . . 8
Tarea C.1.2: Generación de espacio de escalas con imágenes gaussianas . . . 8
Tarea C.1.3: Generación de diferencias de gaussianas . . . 8
Tarea C.1.4: Evaluación de extremos . . . 8
Tarea C.1.5: Localización de puntos clave y orientación de los mismos . . . 9
Tarea C.1.6: Pipeline de generación de puntos clave y descriptores . . . 9
Tarea C.1.7: Correspondencia de características entre imágenes . . . 10
Preguntas . . . 10
C.2: Bolsa de palabras . . . 11
Tarea C.2.1: Carga de los datasets de entrenamiento y validación . . . 11
Tarea C.2.2: Extracción de los descriptores . . . 11
Tarea C.2.3: Creación del vocabulario . . . 11
Tarea C.2.4: Entrenamiento del clasificador . . . 11
Tarea C.2.5: Inferencia en dataset de entrenamiento . . . 11
Tarea C.2.6: Inferencia en dataset de evaluación . . . 12
Preguntas . . . 12

1

Sesión 4: Extracción de características

1.1. Materiales

En esta práctica se trabajará con los siguientes recursos (puede encontrarlos en la sección de Moodle *Laboratorio/Sesión 4*):

- **`partA.ipynb`**: notebook con el código que deberá desarrollar en el apartado A.
- **`partB.ipynb`**: notebook con el código que deberá desarrollar en el apartado B.
- **`partC.ipynb`**: notebook con el código que deberá desarrollar en el apartado C.
- **`helpers`**: archivos con métodos que le ayudarán a obtener los resultados del apartado C.
  - `utils.py`: funciones auxiliares del detector SIFT y de la correspondencia de características.
  - `bow.py`, `dataset.py`, `image_classifier.py` y `results.py`: clases para construir la bolsa de palabras, cargar el *dataset*, entrenar el clasificador y calcular sus resultados.
- **`data`**: carpeta con las imágenes de trabajo (`data/partA-B` para los apartados A y B, y `data/partC` para el apartado C). El *dataset* de la bolsa de palabras se descarga desde Moodle (`dataset.zip`) y se descomprime en `data/partC`.

Los resultados de la práctica se guardan en la carpeta `results`, al mismo nivel que `src` y `data` (desde los notebooks, `../results`).

1.2. Apartados de la práctica

La Sesión 4 del laboratorio está dividida en los siguientes apartados:

- Apartado A: Detección de esquinas (Harris y Shi-Tomasi).
- Apartado B: Detección de líneas rectas (Canny y transformada de Hough).
- Apartado C: Detección de puntos de interés (SIFT) y bolsa de palabras.

1.3. Observaciones

Aunque el guion de la práctica y los comentarios en Markdown del Notebook estén escritos en español, observe que todo aquello que aparece en las celdas de código está escrito en inglés. Es una buena práctica que todo su código esté escrito en inglés.

Aquellas partes del código que deberá completar están marcadas con la etiqueta **`TODO`**.

El apartado C necesita las librerías `tqdm`, `scikit-learn`, `pandas` y `openpyxl`, que no forman parte del entorno del curso. Las primeras celdas de `partC.ipynb` las instalan con `pip`.

Es muy importante que trabaje consultando la documentación de OpenCV[1] para familiarizarse de cara

[1] Documentación de OpenCV: `https://docs.opencv.org/4.x/index.html`

al examen. Tenga en cuenta que en los exámenes no podrá utilizar herramientas de ayuda como Copilot.

1.4. Qué va a aprender

Al finalizar esta práctica, sabrá detectar esquinas con los métodos de Harris y Shi-Tomasi, y extraer líneas rectas con la transformada de Hough. Además, comprenderá cómo funciona por dentro el detector SIFT (espacio de escalas, diferencias de gaussianas y búsqueda de extremos) y lo comparará con la implementación de OpenCV. Por último, construirá una bolsa de palabras visuales para clasificar imágenes de escenas.

1.5. Evaluación

La nota que obtenga en esta sesión de laboratorio será la misma que para ambos miembros de la pareja. Los apartados de la práctica serán evaluados como refleja la Tabla 1.1.

Tarea	Valor	Resultado
Pregunta A.1	1.5	
Pregunta A.2	1.5	
Pregunta B.1	2.0	
Pregunta C.1	2.5	
Pregunta C.2	2.5	
Total	**10.0**	

Table 1.1: Valoración de los apartados de la práctica.

2

Apartado A: **Detección de esquinas**

En este apartado detectará las esquinas de las imágenes de la carpeta `data/partA-B` (*football*, *geometry*, *sudoku* y *tennis*) con los métodos de Harris y Shi-Tomasi. El código se desarrolla en el notebook `partA.ipynb`.

Tarea A.1: Creación de carpeta de resultados

Cree una nueva carpeta llamada `partA_processed` dentro de la carpeta `results` (desde el notebook, `../results/partA_processed`), con el objetivo de guardar en ella los resultados de este apartado.

Tarea A.2: Carga de imágenes

Defina y ejecute los dos métodos propuestos para cargar imágenes, `imageio_load_images()` y `opencv_load_images()`, a partir de la lista ordenada de rutas de las imágenes de `data/partA-B`, que obtendrá con `glob`. Guarde la última imagen de cada lista (la pista de tenis) usando la misma función `cv2.imwrite()` y observe lo que ocurre.

Tarea A.3: Definición del método de Harris para detección de esquinas

Defina la función `harris_corner_detector()`, que servirá para detectar las esquinas de las imágenes de trabajo. Siga los siguientes pasos:

- Convertir la imagen de entrada a escala de grises usando `cv2.cvtColor()`.
- Transformar la imagen en escala de grises al tipo `float32` con `np.float32()`.
- Aplicar el método `cv2.cornerHarris()` [1].
- Dilatar la imagen resultante con `cv2.dilate()` para mejorar la visibilidad de las esquinas detectadas.
- Resaltar en rojo los píxeles cuya respuesta supere el 1% del valor máximo.

Tarea A.4: Aplicación de Harris

Aplique la función `harris_corner_detector()` sobre las imágenes *geometry* y *football*, ajustando sus parámetros para cada una. Guarde los resultados en `../results/partA_processed` con el nombre `Harris_{save_name}.jpg`, como se indica en los comentarios del notebook. La Figura 2.1 muestra un ejemplo del resultado que debería obtener.

[1] Documentación del método Harris en OpenCV:
https://docs.opencv.org/3.4/dd/d1a/group__imgproc__feature.html#gac1fc3598018010880e370e2f709b4345

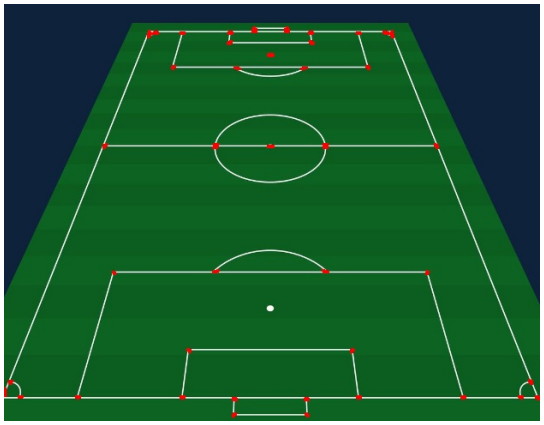


Figure 2.1: Ejemplo de detección de esquinas con el método de Harris.

Tarea A.5: Definición del método de Shi-Tomasi para detección de esquinas

Defina la función `shi_tomasi_corner_detection()`, que servirá para detectar las esquinas de las imágenes de trabajo. Siga los siguientes pasos:

- Convertir la imagen de entrada a escala de grises utilizando `cv2.cvtColor()`.
- Aplicar el método `cv2.goodFeaturesToTrack()` [2].
- Convertir las coordenadas de las esquinas detectadas a enteros utilizando `np.intp()`.
- Para cada esquina detectada, extraer las coordenadas `x` e `y` y dibujar un círculo en la imagen usando `cv2.circle()`.

Tarea A.6: Aplicación de Shi-Tomasi

Aplique la función `shi_tomasi_corner_detection()` sobre las imágenes *geometry* (esquinas en morado) y *football* (esquinas en naranja), con círculos de radio 4. Tenga en cuenta el espacio de color en el que trabaja OpenCV al definir los colores. Guarde los resultados en `../results/partA_processed` con el nombre `Shi-Tomasi_{save_name}.jpg`, como se indica en los comentarios del notebook.

Preguntas

Pregunta A.1

Realice la detección de esquinas en las otras dos imágenes de la carpeta `data/partA-B` (cuyos nombres de guardado han de ser *sudoku* y *tennis*) con el método de Harris.

Pregunta A.2

Realice la detección de esquinas en las otras dos imágenes de la carpeta `data/partA-B` (cuyos nombres de guardado han de ser *sudoku* y *tennis*) con el método de Shi-Tomasi.

[2] Documentación del método `goodFeaturesToTrack()` en OpenCV:
https://docs.opencv.org/3.4/dd/d1a/group__imgproc__feature.html#ga1d6bb77486c8f92d79c8793ad995d541

3

Apartado B: **Detección de líneas rectas**

En este apartado detectará las líneas rectas de las imágenes de la carpeta `data/partA-B` (las mismas del apartado A) combinando el detector de bordes de Canny con la transformada de Hough. En las tareas trabajará con dos de las imágenes y, en la pregunta, repetirá el proceso con las dos restantes. El código se desarrolla en el notebook `partB.ipynb`. Puede guardar o visualizar los resultados como prefiera (vea la nota del notebook).

Tarea B.1: Aplicación de Canny

Para obtener los bordes de las imágenes, aplique el método `cv2.Canny()` [1] de OpenCV a las imágenes de trabajo ajustando los hiperparámetros.

Recuerde que para aplicar Canny, primero deberá pasar su imagen a escala de grises con el método `cv2.cvtColor()`. Por otro lado, observe que el método Canny devuelve una imagen binaria (blanco y negro) con los bordes detectados.

Tarea B.2: Visualización de líneas

Implemente la función `draw_lines()` para pintar las líneas detectadas sobre las imágenes. Cada línea viene dada por las coordenadas de sus dos extremos (x_1, y_1, x_2, y_2). Para pintarlas, puede utilizar el método `cv2.line()` [2] de OpenCV.

Tarea B.3: Aplicación de la transformada de Hough

Aplique el método `cv2.HoughLinesP()` [3] de OpenCV a las imágenes de bordes obtenidas en la Tarea B.1 y afine los hiperparámetros para extraer las líneas de interés. Pinte las líneas sobre una copia de la imagen original, ya que `draw_lines()` modifica la imagen que recibe. La Figura 3.1 muestra el resultado esperado al aplicar la transformada de Hough sobre una de las imágenes.

[1] Documentación del método `cv2.Canny()` en OpenCV:
https://docs.opencv.org/3.4/dd/d1a/group__imgproc__feature.html#ga04723e007ed888ddf11d9ba04e2232de

[2] Documentación del método `cv2.line()` en OpenCV:
https://docs.opencv.org/3.4/d6/d6e/group__imgproc__draw.html#ga7078a9fae8c7e7d13d24dac2520ae4a2

[3] Documentación del método `cv2.HoughLinesP()` en OpenCV:
https://docs.opencv.org/3.4/dd/d1a/group__imgproc__feature.html#ga8618180a5948286384e3b7ca02f6feeb



Figure 3.1: Ejemplo de detección de líneas con la transformada de Hough.

Preguntas

Pregunta B.1

Repita el procedimiento para extraer las líneas de las dos imágenes restantes.

4

Apartado C: **Detección de puntos de interés y bolsa de palabras**

En este apartado deberá extraer los puntos de interés de la imagen `data/partC/0.jpg`. Una vez hecho, podrá comprobar la correspondencia con esa misma imagen girada (`data/partC/0_rot.jpg`) y la relación entre ambas (Figura 4.1). Tras entender el funcionamiento de este método, pasará a construir una bolsa de palabras a partir de un *dataset* de entrenamiento y a generar predicciones para un *dataset* de validación. El código se desarrolla en el notebook `partC.ipynb`.

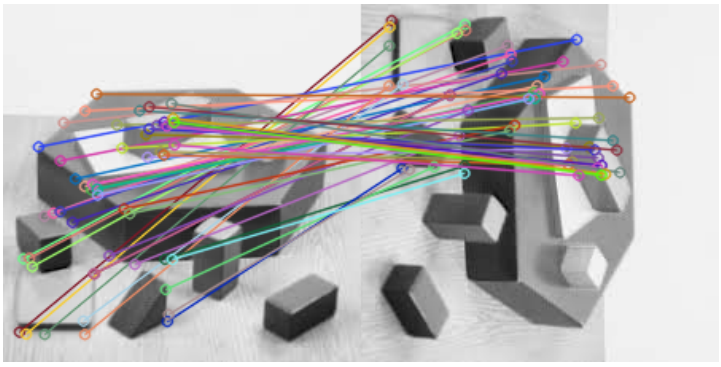


Figure 4.1: Correspondencia de características entre una imagen y la misma girada, con el SIFT de OpenCV.

Tarea PREVIA: Creación de las carpetas de resultados

Cree la carpeta `../results/partC` con las subcarpetas `bow` y `feature_extraction`. Dentro de esta última, cree las subcarpetas `0_gaussians`, `1_difference_of_gaussians`, `2_match_features` y `3_SIFT`.

C.1: Detección de puntos de interés

En esta sección se desarrollarán las funciones necesarias para detectar los puntos de interés y calcular sus descriptores empleando la transformación SIFT. La transformación SIFT consta de cuatro pasos principales:

1. Detección de extremos en el espacio de escalas.
2. Localización de los puntos de interés.
3. Asignación de orientaciones.
4. Generación de los descriptores de los puntos de interés.

Cada uno se irá desarrollando con las funciones necesarias para completarlo. Se comenzará generando el espacio de escalas en el que se buscarán los extremos.

Tarea C.1.1: Generar función de filtro gaussiano

Para generar el espacio de escalas que se desea analizar, se debe aplicar un filtrado gaussiano a la imagen original hasta obtener un número de imágenes gaussianas relevante. Para ello, debe desarrollar la función `generateGaussianImages()` empleando el método `cv2.GaussianBlur()` [1] de la librería OpenCV.

Como entrada, se recibe una imagen y una lista de sigmas que deberá aplicar en el filtrado. Deberá generar tantas imágenes gaussianas como sigmas haya en la lista, aplicando cada filtro al resultado del filtrado anterior (y a la imagen original en el primer caso).

Tarea C.1.2: Generación de espacio de escalas con imágenes gaussianas

Deberá cargar la imagen base (`data/partC/0.jpg`) con cv2 y en escala de grises, sin emplear la función `cv2.cvtColor()`. A continuación se definen los parámetros iniciales del espacio de escalas: la *sigma* inicial y el número de diferencias de gaussianas que se desea generar (`intervals`). Aunque el mínimo de diferencias necesarias para detectar los máximos y mínimos en ventanas de 3x3x3 es 3, con ese valor el refinamiento de la localización (Tarea C.1.5) descarta todos los puntos de interés de esta imagen; por eso el notebook usa `intervals = 5`. Por último, emplee el método `generateGaussianImages()` que ha desarrollado en la tarea anterior.

Como puede observar en el código, se le proporciona el método `generateGaussianSigmas()`, que recibe la *sigma* inicial y el número de diferencias de gaussianas que desea generar. Este método devuelve la lista de valores de *sigma* que deberá aplicar en los filtrados consecutivos: si quiere generar *n* diferencias de gaussianas, le devolverá *n+1* valores de sigma. La Figura 4.2 muestra el resultado esperado de esta tarea.

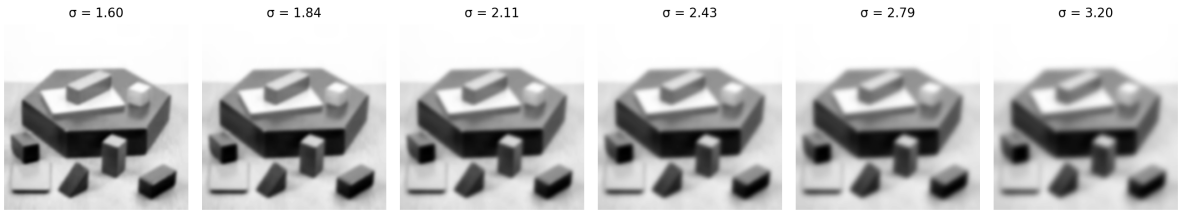


Figure 4.2: Resultado esperado de la generación de imágenes gaussianas (*sigma* total acumulada en cada imagen).

Tarea C.1.3: Generación de diferencias de gaussianas

A partir de la lista de imágenes anteriores, se deberá generar una nueva lista con las diferencias entre imágenes consecutivas empleando el método `cv2.subtract()` [2] de la librería OpenCV. La Figura 4.3 muestra el resultado esperado de esta tarea.

Tarea C.1.4: Evaluación de extremos

En esta tarea debe desarrollar la función `isPixelAnExtremum()`, que evalúa si el píxel central de una ventana de 3x3x3 es un máximo o un mínimo entre sus vecinos y si supera un umbral mínimo en valor absoluto. La ventana está compuesta por 3 regiones de 3x3 píxeles, que corresponden a regiones de 3 diferencias de gaussianas consecutivas. La función debe devolver *True* en caso de ser un máximo o un mínimo y *False* en cualquier otro caso. **Importante:** las diferencias de gaussianas pueden tomar valores negativos.

[1] Documentación del método `cv2.GaussianBlur()` en OpenCV:
https://docs.opencv.org/3.4/d4/d86/group__imgproc__filter.html#gaabe8c836e97159a9193fb0b11ac52cf1

[2] Documentación del método `cv2.subtract()` en OpenCV:
https://docs.opencv.org/3.4/d2/de8/group__core__array.html#gaa0f00d98b4b5edeaeb7b8333b2de353b



Figure 4.3: Resultado esperado de la generación de diferencias de gaussianas (valores reescalados para su visualización).

Tarea C.1.5: Localización de puntos clave y orientación de los mismos

En esta tarea deberá completar la función `findScaleSpaceExtrema()`, que emplea la función de la tarea anterior.

Primero tendrá que completar el bucle `for` para obtener en cada iteración 3 diferencias de gaussianas consecutivas y enumerarlas. Tras esto, deberá iterar sobre las coordenadas de la imagen teniendo en cuenta que la ventana de 3x3 debe recorrer toda la imagen sin que ninguno de sus puntos quede fuera de ella. Complete la llamada a la función `isPixelAnExtremum()` con los argumentos pertinentes para evaluar si el píxel en las coordenadas de esa iteración es un extremo.

Los píxeles que son máximos o mínimos dan una localización aproximada de los puntos de interés de la imagen. La función `localizeExtremumViaQuadraticFit()` (ya proporcionada) refina esta localización teniendo en cuenta el gradiente y el hessiano, y descarta los puntos que no tienen suficiente contraste. Con la localización final, se obtiene la orientación del punto de interés gracias a la función `computeKeypointsWithOrientations()` (también proporcionada). Con ello, el punto de interés queda completamente definido.

Se recomienda el uso de la función de borrado de duplicados `removeDuplicateKeypoints()` (también proporcionada), ya que el proceso anterior genera algunos puntos de interés repetidos. Para visualizar los puntos de interés sobre la imagen original se proporciona la función `visualizeKp()`. La Figura 4.4 muestra el resultado que debería obtener.

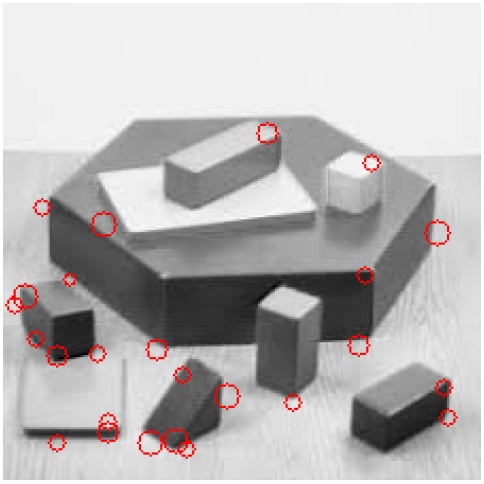


Figure 4.4: Resultado esperado de la detección de puntos de interés y su orientación.

Una vez obtenidas las imágenes gaussianas y los puntos de interés con sus orientaciones, se pueden generar los descriptores de las regiones vecinas a estos, que son invariantes a los cambios de iluminación y de punto de vista. En esta sesión se proporciona para ello la función `generateDescriptors()`.

Tarea C.1.6: Pipeline de generación de puntos clave y descriptores

En esta tarea deberá completar la función `computeKeypointsAndDescriptors()`. Esta función recibe la imagen original, el valor inicial de *sigma* y el número de diferencias de gaussianas que se desea generar.

Complétela con los métodos desarrollados en las tareas anteriores para obtener el *pipeline* completo de localización de puntos de interés y generación de descriptores.

Tarea C.1.7: Correspondencia de características entre imágenes

En esta tarea comprobará la capacidad del sistema para localizar las mismas características en una imagen alterada y establecer la correspondencia entre las características de la imagen original y las de la alterada. Para ello, cargue la imagen original y la alterada (`0.jpg` y `0_rot.jpg`) en escala de grises con `cv2` y haga las llamadas pertinentes al método de la tarea anterior para obtener los puntos de interés y los descriptores de ambas imágenes.

Con estos datos, la función `matchFeatures_and_save()` establece la correspondencia entre los puntos de interés de ambas imágenes, la muestra y la guarda en la ruta indicada (por ejemplo, en `../results/partC/feature_extraction/2_match_features`). Si solo quiere visualizarla, puede usar `matchFeatures()`. Ambas necesitan más de 10 correspondencias para dibujar el resultado. La Figura 4.5 muestra el resultado esperado.

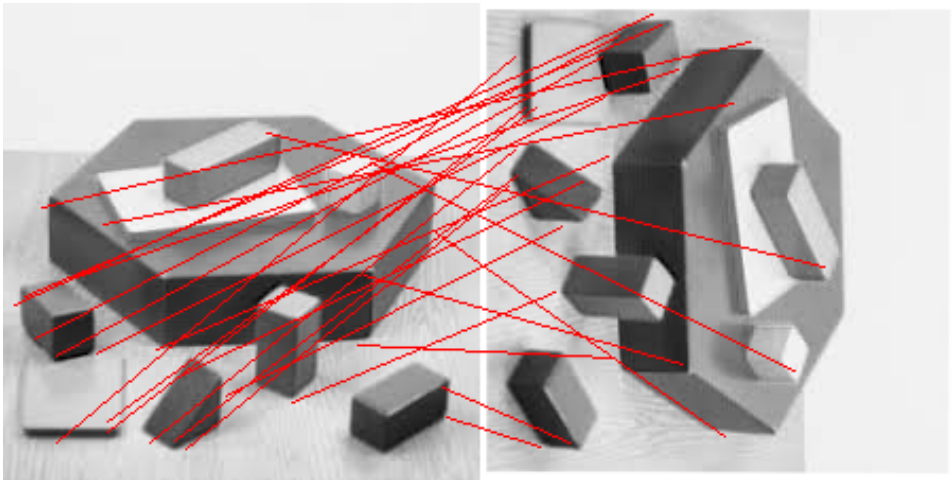


Figure 4.5: Resultado esperado de la correspondencia de características con el *pipeline* desarrollado.

Una vez comprendido el funcionamiento de un detector de puntos de interés y de sus descriptores, se le anima a emplear el método propio de OpenCV, como se proporciona en el notebook (*Ampliación: SIFT con OpenCV*).

Preguntas

Pregunta C.1: Correspondencia de imágenes propias y evaluación

Con los métodos desarrollados para la extracción de características, use su *pipeline* y el SIFT de OpenCV con sus propias imágenes (por ejemplo, dos fotos de un mismo objeto desde puntos de vista distintos). Adjunte los resultados de ambos y compárelos extrayendo sus propias conclusiones.

C.2: Bolsa de palabras

En esta sección se va a generar una bolsa de palabras a partir de un *dataset* de entrenamiento y se evaluará con un *dataset* de validación. Para generar la bolsa de palabras, se seguirán los pasos descritos en las sesiones de teoría:

1. Extracción de los descriptores de los puntos de interés de las imágenes del *dataset*.
2. Construcción del diccionario de palabras visuales (K-Means).
3. Para cada imagen del *dataset*, generación del histograma de ocurrencias de las palabras del diccionario.
4. Entrenamiento de un clasificador con los histogramas de las imágenes del *dataset* (SVM).

Tarea C.2.1: Carga de los datasets de entrenamiento y validación para la bolsa de palabras

Descargue el *dataset* desde Moodle (`dataset.zip`) y descomprímalo en `data/partC`, de forma que queden las carpetas `data/partC/dataset/training` y `data/partC/dataset/validation`. Contiene imágenes de 15 tipos de escenas (*Coast*, *Forest*, *Kitchen*, *Street*, etc.), organizadas en una carpeta por categoría.

Como ayuda, se le proporciona la clase de Python `Dataset`. Empleando su método `load()`, cargue los *datasets* de entrenamiento y de validación.

Tarea C.2.2: Extracción de los descriptores

Una vez cargados los *datasets*, se pueden extraer los descriptores de las imágenes que poblarán la bolsa de palabras. Para ello, cargue en escala de grises cada imagen del *dataset* de entrenamiento a partir de su ruta. Después, empleando los métodos de OpenCV que ha visto durante la sesión, obtenga sus descriptores SIFT y guárdelos en una lista.

Tarea C.2.3: Creación del vocabulario

Con los descriptores extraídos, se puede proceder a agruparlos en las distintas palabras del vocabulario con el algoritmo de clusterización K-Means. Agrupar los cerca de 1.5 millones de descriptores del *dataset* es lento y consume mucha memoria, así que el notebook toma una muestra aleatoria de 70 descriptores por imagen (unos 200 000 en total), que produce un vocabulario igual de útil. En esta tarea debe añadir esa muestra a la bolsa de palabras `words` que se inicializa en el código, sobre la que se ejecutará el algoritmo. Fíjese en que el número de palabras del vocabulario se determina con la variable `vocabulary_size`, y en que esto afectará al desempeño del clasificador más adelante. El vocabulario se guarda en `../results/partC/bow/vocabulary.pickle`.

Tarea C.2.4: Entrenamiento del clasificador

Una vez agrupadas las características de las imágenes en palabras, se va a entrenar un clasificador capaz de distinguir las categorías del *dataset* a partir de los histogramas de palabras de cada imagen. Será un modelo basado en una *Support Vector Machine* (SVM). Con este clasificador se podrán clasificar imágenes externas al *dataset* de entrenamiento.

Para entrenar el clasificador se aportan dos clases como ayuda: `BoW`, que carga el vocabulario, e `ImageClassifier`, que implementa el clasificador SVM. Añada los argumentos necesarios a los métodos especificados en la celda de código; compruebe qué argumentos debe añadir consultando el código de las clases proporcionadas. El modelo final se guarda en `../results/partC/bow` para usarlo más adelante.

Tarea C.2.5: Inferencia en dataset de entrenamiento

Con el modelo entrenado, es hora de comprobar su desempeño. La métrica empleada será la precisión (*accuracy*) y podrá observar la matriz de confusión de las predicciones, que además se guarda en un archivo Excel en `../results/partC/bow`.

Al igual que en la tarea anterior, añada los argumentos necesarios a los métodos especificados en la celda de código.

Tarea C.2.6: Inferencia en dataset de evaluación

Después de obtener los resultados con el *dataset* de entrenamiento, se deben comparar con los del *dataset* de validación, con el que se debería obtener un desempeño inferior.

Al igual que en la tarea anterior, añada los argumentos necesarios.

Preguntas

Pregunta C.2: Búsqueda de los mejores parámetros. ¿Cuántas palabras usar?

Entrene y evalúe el clasificador con vocabularios de 50, 100, 200 y 400 palabras (`vocabulary_size`). Elija además **uno** de estos parámetros y estudie su efecto: el kernel del SVM (`setKernel()` en `image_classifier.py`), el número de iteraciones del SVM (argumento `iterations` de `train()`) o el extractor de características (KAZE en lugar de SIFT). Muestre en una tabla la *accuracy* de entrenamiento y de validación de cada prueba y justifique qué configuración elegiría.